NAME :- Lokare Mahir Abdul Majeed

ROLL NO :- 23AI34

BATCH :- 02

EXPERIMENT NO :- 03

AIM :-

Train a Deep Neural Network using backpropagation with two hidden layers.

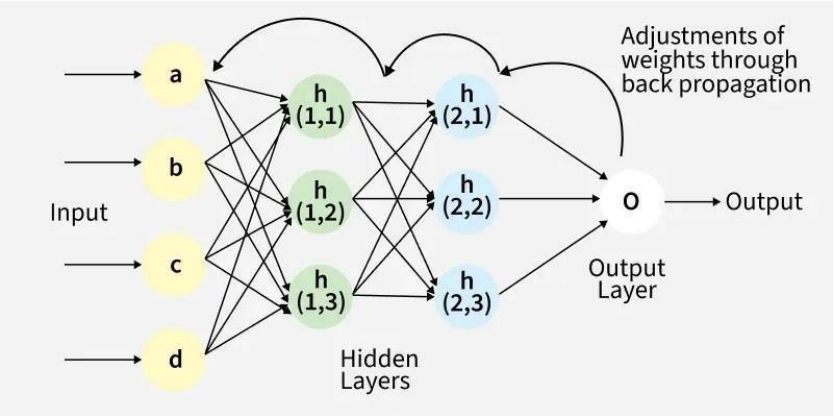
OBJECTIVE :-

1. To understand the concept of backpropagation in Deep Neural Networks.
2. To implement a neural network with two hidden layers.
3. To understand forward propagation and backward propagation.
4. To calculate gradients using the chain rule.
5. To update weights and biases using gradient descent.
6. To observe the reduction in loss during training.

THEORY :-

Deep Neural Network :-

A Deep Neural Network (DNN) is an artificial neural network containing multiple layers between the input and output layers. In this experiment, the network contains two hidden layers. The architecture used is:



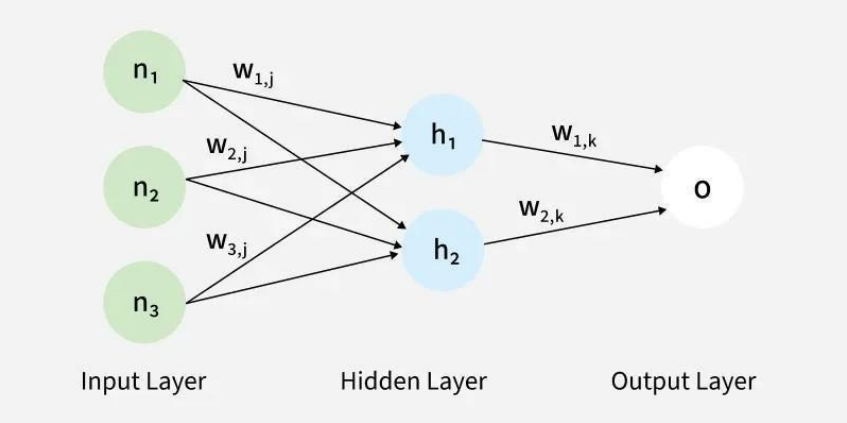
Working of Back Propagation Algorithm :-

The Back Propagation algorithm involves two main steps:

1. Forward Pass

In the forward pass, input data moves through the network to generate an output.

1. Input data is passed to the input layer and forwarded to the hidden layers.
2. Each neuron computes a weighted sum and adds a bias.
3. Activation functions such as ReLU are applied to produce outputs.
4. Outputs from one layer become inputs to the next layer.
5. The final layer produces the prediction.



2. Backward Pass

In the backward pass, the error between the predicted and actual output is propagated backward through the network to update weights and biases.

The error can be calculated using Mean Squared Error (MSE): MSE = (Predicted Output − Actual Output)²

The main steps are:

1. Gradients are computed using the chain rule.

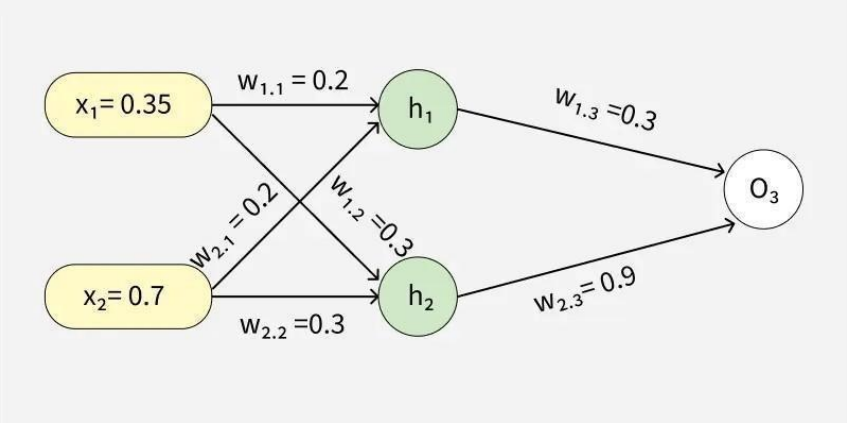
2. Weights and biases are updated to reduce the error.

3. The process continues layer by layer from the output layer toward the input layer.

4. Derivatives of activation functions are used to calculate the gradients.

Example :-

Consider a simple neural network using the sigmoid activation function. Assume that the target output is 0.5 and the learning rate is 1.



Forward Propagation

1. Initial Calculation

The weighted sum at each node is calculated using: aj = Σ(wi,j × xi)

Where: a_j is the weighted sum of inputs and weights at neuron j; $w_{i,j}$ represents the weight between the ith input and jth neuron; x_i represents the value of the ith input.

After applying the activation function: o_j = Activation Function(a_j)

2. Sigmoid Function

The sigmoid function produces an output between 0 and 1.

$$y_j = \frac{1}{1+e^{-a_j}}$$

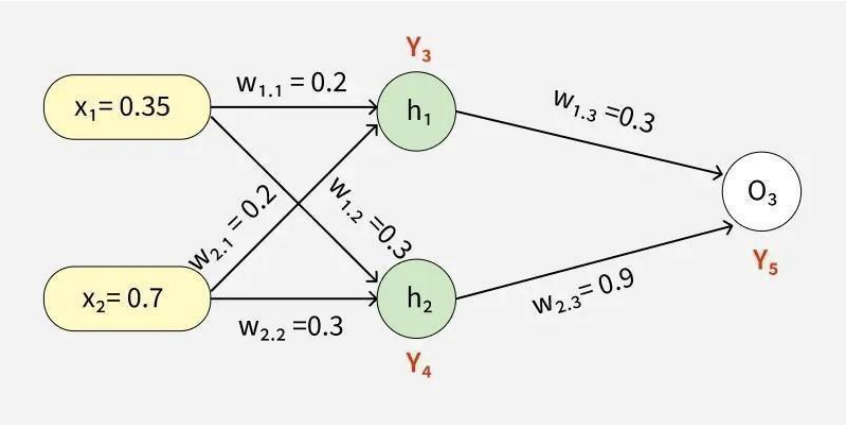
3. Computing Outputs

For hidden node h1: a1 = (w1,1 × x1) + (w2,1 × x2) = (0.2 × 0.35) + (0.2 × 0.7) = 0.21

Therefore: y3 = 1 / (1 + $e^{-0.21}$) ≈ 0.56

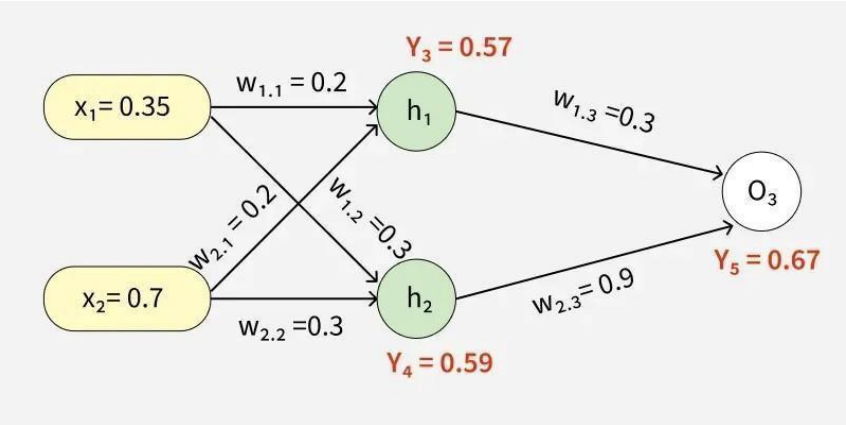
Similarly, for hidden node h2: a2 = (w1,2 × x1) + (w2,2 × x2) = (0.3 × 0.35) + (0.3 × 0.7) = 0.315

Therefore: y4 = 1 / (1 + $e^{-0.315}$) ≈ 0.59



For the output node: a3 = (w1,3 × y3) + (w2,3 × y4) = (0.3 × 0.57) + (0.9 × 0.59) = 0.702

Therefore: y5 = 1 / (1 + $e^{-0.702}$) ≈ 0.67



4. Error Calculation

The actual target output is 0.5, while the obtained output is 0.67.

Error = y_target − y5 = 0.5 − 0.67 = −0.17

This error is then propagated backward through the network.

Back Propagation

1. Calculating Gradients

The change in each weight is calculated using: Δwi = η × δ × Oi, where δ is the error term for the neuron, η is the learning rate, and Oi is the output of the previous neuron.

2. Output Unit Error

For output unit O3: δ5 = y5(1 − y5)(y_target − y5) = 0.67(1 − 0.67)(−0.17) ≈ −0.0376

3. Hidden Unit Error

For hidden unit h1: $\delta 3 = y3(1 - y3)(w1,3 \times \delta 5) = 0.56(1 - 0.56)(0.3 \times -0.0376) \approx -0.0027$

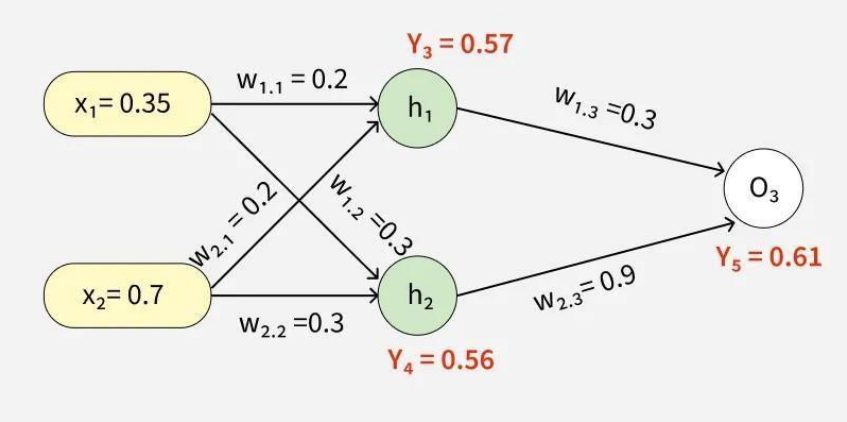
For hidden unit h2: $\delta 4 = y4(1 - y4)(w2,3 \times \delta 5) = 0.59(1 - 0.59)(0.9 \times -0.0376) \approx -0.00819$

4. Weight Updates

For the weights from the hidden layer to the output layer: $\Delta w2,3 = 1 \times (-0.0376) \times 0.59 = -0.022184$. New weight: $w2,3(new) = 0.9 - 0.022184 = 0.877816$

For the weights from the input layer to the hidden layer: $\Delta w1,1 = 1 \times (-0.0027) \times 0.35 = -0.000945$. New weight: $w1,1(new) = 0.2 - 0.000945 = 0.199055$

Similarly, the other weights are updated: w1,2(new) = 0.273225, w1,3(new) = 0.086615, w2,1(new) = 0.269445, w2,2(new) = 0.18534



After updating the weights, the forward pass is repeated and produces approximately: y3 = 0.57, y4 = 0.56, y5 = 0.61

Since y5 = 0.61 is still different from the target output of 0.5, the process of calculating the error and backpropagating continues.

PROCEDURE :-

1. Import NumPy and Matplotlib.
2. Generate a training dataset.
3. Normalize the input data.
4. Create a DNN containing two hidden layers.
5. Initialize the weights and biases using He initialization.
6. Perform forward propagation through both hidden layers.
7. Calculate the MSE loss.
8. Perform backpropagation using the chain rule.
9. Calculate gradients for all weights and biases.
10. Update the parameters using gradient descent.
11. Repeat the process for multiple epochs.
12. Record the loss after every epoch.
13. Plot the training loss curve.
14. Evaluate the trained model on test data.

CODE :-

```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(42)

m = 500
X = np.random.randn(m, 2)

true_W1 = np.array([[1.5, -2.0], [0.5, 1.0]])
true_b1 = np.array([0.5, -0.5])
true_W2 = np.array([[2.0], [-1.5]])
true_b2 = 1.0

h1 = np.maximum(0, X @ true_W1 + true_b1)
h2 = np.maximum(0, h1 @ true_W2 + true_b2)
y = h2 + 0.05 * np.random.randn(m, 1)

X = (X - X.mean(axis=0)) / X.std(axis=0)

class DNN:
    def __init__(self, dims):
        self.dims = dims
        self.L = len(dims) - 1
        self.params = {}
        for l in range(1, self.L + 1):
            self.params[f"W{l}"] = np.random.randn(dims[l - 1], dims[l]) * np.sqrt(2 /
dims[l - 1])
            self.params[f"b{l}"] = np.zeros((1, dims[l]))
        self.cache = {}

    def relu(self, Z):
        return np.maximum(0, Z)

    def relu_deriv(self, Z):
        return (Z > 0).astype(float)

    def forward(self, X):
        A = X
        self.cache["A0"] = X
        for l in range(1, self.L + 1):
            Z = A @ self.params[f"W{l}"] + self.params[f"b{l}"]
            self.cache[f"Z{l}"] = Z
            if l == self.L:
                A = Z
            else:
                A = self.relu(Z)
            self.cache[f"A{l}"] = A
        return A

    def backward(self, X, y, y_pred):
        m = X.shape[0]
        grads = {}
        dZ = (y_pred - y) / m
        grads[f"dW{self.L}"] = self.cache[f"A{self.L - 1}"].T @ dZ
        grads[f"db{self.L}"] = np.sum(dZ, axis=0, keepdims=True)

        for l in range(self.L - 1, 0, -1):
            dA = dZ @ self.params[f"W{l + 1}"].T
            dZ = dA * self.relu_deriv(self.cache[f"Z{l}"])
            grads[f"dW{l}"] = self.cache[f"A{l - 1}"].T @ dZ
            grads[f"db{l}"] = np.sum(dZ, axis=0, keepdims=True)
        return grads
```

```
    def update(self, grads, lr=0.01):
        for l in range(1, self.L + 1):
            self.params[f"W{l}"] -= lr * grads[f"dW{l}"]
            self.params[f"b{l}"] -= lr * grads[f"db{l}"]

    def train_epoch(self, X, y, batch_size=32, lr=0.01):
        m = X.shape[0]
        indices = np.random.permutation(m)
        total_loss = 0
        batches = 0
        for i in range(0, m, batch_size):
            idx = indices[i:i + batch_size]
            X_batch = X[idx]
            y_batch = y[idx]
            y_pred = self.forward(X_batch)
            grads = self.backward(X_batch, y_batch, y_pred)
            self.update(grads, lr)
            loss = np.mean((y_pred - y_batch) ** 2)
            total_loss += loss
            batches += 1
        return total_loss / batches

dims = [X.shape[1], 8, 6, 1]
model = DNN(dims)
losses = []
epochs = 150
learning_rate = 0.01

for epoch in range(epochs):
    loss = model.train_epoch(X, y, batch_size=32, lr=learning_rate)
    losses.append(loss)
    if (epoch + 1) % 25 == 0:
        print(f"Epoch {epoch + 1}: Loss = {loss:.6f}")

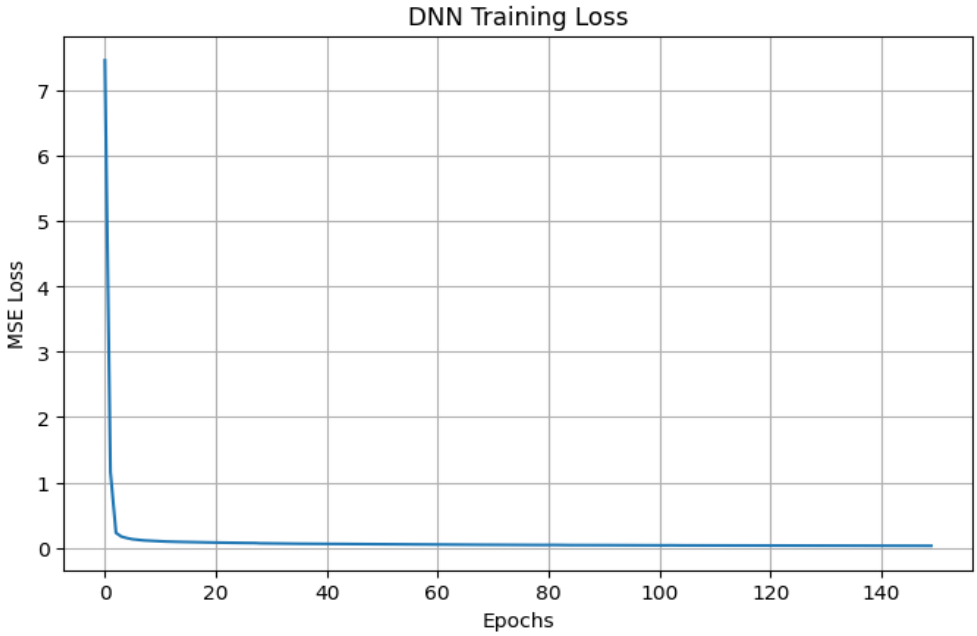
test_X = np.random.randn(20, 2)
test_X = (test_X - X.mean(axis=0)) / X.std(axis=0)
test_predictions = model.forward(test_X)
print("\nTest predictions shape:", test_predictions.shape)

plt.figure(figsize=(8, 5))
plt.plot(losses)
plt.xlabel("Epochs")
plt.ylabel("MSE Loss")
plt.title("DNN Training Loss")
plt.grid(True)
plt.show()
```

OUTPUT :-

```
Epoch 25: Loss = 0.073860
Epoch 50: Loss = 0.053981
Epoch 75: Loss = 0.044224
Epoch 100: Loss = 0.036822
Epoch 125: Loss = 0.032348
Epoch 150: Loss = 0.028792

Test predictions shape: (20, 1)
```



CONCLUSION :-

The experiment successfully trained a Deep Neural Network with two hidden layers using backpropagation. The network calculated gradients through the chain rule and updated its weights and biases to reduce the MSE loss. The decreasing loss during training demonstrates that the network learned the underlying relationship in the dataset.